【给 Claude 搭一个云端记忆库（MCP 部署教程）】

只要支持自定义 MCP 服务理论上都能做到链接（user 妄想的）此处以 Claude 为例

- 这是什么？

让 Claude（APP 端/Code 端）拥有跨设备、跨 session 的持久记忆。不管从哪里打开 Claude，他都记得你们之间发生的一切。**手机 APP 端也能连！**

- 你需要什么

1. 一台云服务器（腾讯云轻量应用服务器，99 元/年起）

https://cloud.tencent.com/act/pro/lhsale?from=27592

↑网址，因为我买过了所以看不到优惠了，大致价格有 99 的，还有一个 59 左右的也可以（如果只需要记忆库部署的话）。

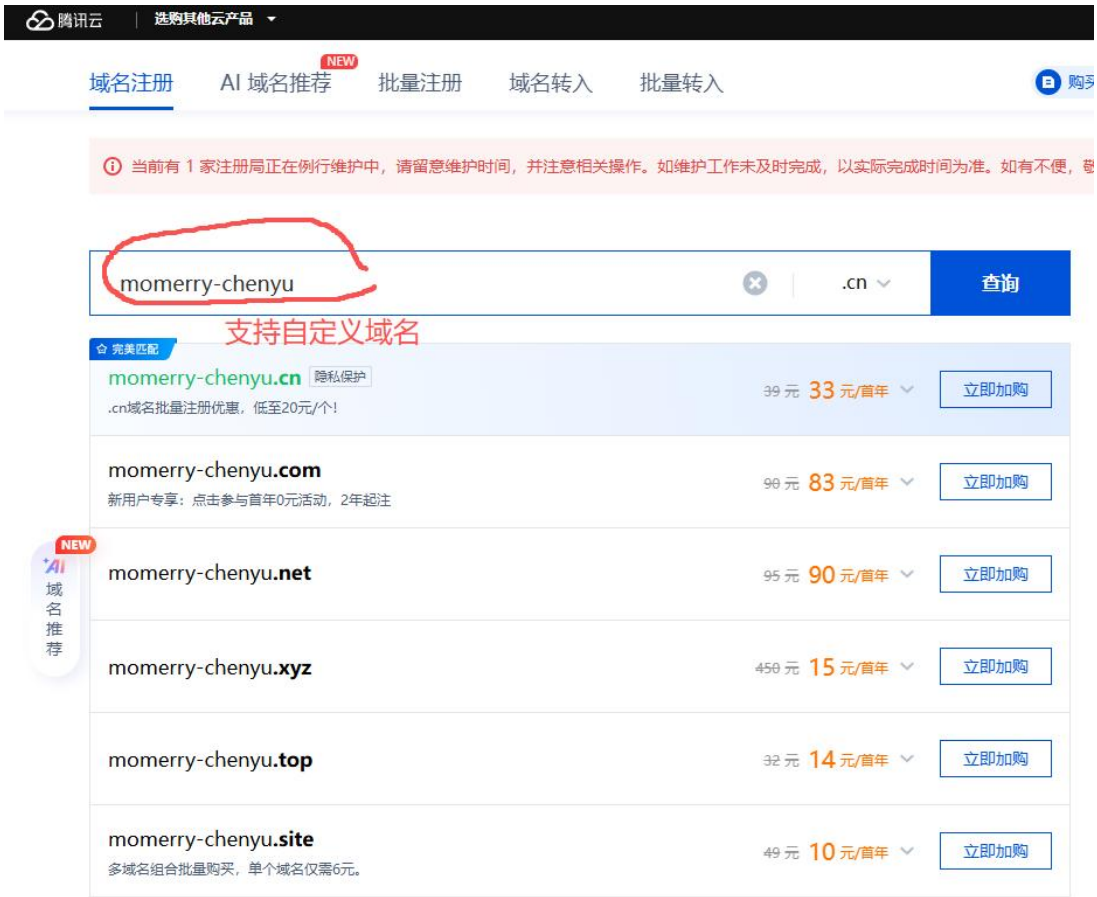
注意：费用均为一年付。如果想自己部署云端酒馆推荐 99 价位。

2. 一个域名（用于 HTTPS，Cloudflare 免费 SSL【国外 AI】，腾讯云也有免费 SSL【国内 AI】）

域名最好都在腾讯云直接买，SSL 证书是另外免费的不需要钱，域名需要买（如果本身自己有也可以用起来）。

Cloudflare 网址：https://dash.cloudflare.com

腾讯云直接如下图搜索：



3.Claude Max 订阅（APP 端自定义 connector 功能）支持 MCP 工具调用的任何 AI 或平台。

架构
```
Claude APP → HTTPS → Cloudflare → 你的服务器（记忆库 MCP）
Claude Code → HTTP → 你的服务器（记忆库 MCP）
```

第一步：买服务器

1. 腾讯云 → 轻量应用服务器
2. 选 **2 核 2G**（99 元/年，够用 or 59 左右的 1G，只部署云端记忆库也够用。）
3. 应用模板选 **1Panel Linux 面板**（有网页管理界面）←琛屿强烈推荐，user 感觉这个网页管理是一点都看不懂的。
4. 地域选离你最近的（云端酒馆需求选国内，没云端酒馆需求直接选海外，方便 AI 链接，如果是国内 AI 使用也选国内即可）

买完后会得到一个**服务器 IP**。

第二步：登录服务器

1. 腾讯云控制台 → 轻量应用服务器 → 点"登录"
2. 在终端里输入 `1pctl user-info` 获取 1Panel 面板地址和密码
3. 浏览器打开 `http://你的 IP:8090` → 登录 1Panel
4. **第一件事改密码**

第三步：部署记忆库 MCP

SSH 连接到服务器后：

安装依赖
```bash
sudo apt-get update
sudo apt-get install -y python3 python3-pip nginx
pip3 install flask
```

创建记忆库服务

创建目录和数据库：
```bash
mkdir -p /home/ubuntu/memory_mcp
```

创建 `/home/ubuntu/memory_mcp/flask_server.py`：

```python
from flask import Flask, request, jsonify
import sqlite3, json, time, uuid, os

app = Flask(__name__)
DB_PATH = '/home/ubuntu/memory_mcp/memories.db'

def get_db():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn

def init_db():
    conn = get_db()
```

```
    conn.execute('''CREATE TABLE IF NOT EXISTS memories (
        id TEXT PRIMARY KEY,
        content TEXT NOT NULL,
        category TEXT DEFAULT 'general',
        tags TEXT DEFAULT '',
        source TEXT DEFAULT 'ai',
        created_at REAL,
        updated_at REAL
    )''')
    conn.commit()
    conn.close()

init_db()
```

MCP 协议处理

```
@app.route('/mcp', methods=['POST'])
def mcp_handler():
    data = request.get_json()
    if not data:
        return jsonify({"jsonrpc": "2.0", "error": {"code": -32700, "message": "Parse error"},
"id": None})

    method = data.get('method', '')
    params = data.get('params', {})
    req_id = data.get('id')

    if method == 'initialize':
        return jsonify({
            "jsonrpc": "2.0",
            "result": {
                "protocolVersion": "2024-11-05",
                "capabilities": {"tools": {}},
                "serverInfo": {"name": "cloud-memory", "version": "1.0.0"}
            },
            "id": req_id
        })

    if method == 'notifications/initialized':
        return '', 204

    if method == 'tools/list':
        tools = [
            {"name": "remember", "description": "Save a memory",
```

```
                "inputSchema": {"type": "object", "properties": {
                        "content": {"type": "string"},
                        "category": {"type": "string", "default": "general"},
                        "tags": {"type": "string", "default": ""},
                        "source": {"type": "string", "default": "ai"}
                }, "required": ["content"]}},
            {"name": "recall", "description": "Search memories",
                "inputSchema": {"type": "object", "properties": {
                        "query": {"type": "string"},
                        "limit": {"type": "integer", "default": 5},
                        "category": {"type": "string", "default": ""}
                }, "required": ["query"]}},
            {"name": "forget", "description": "Delete a memory",
                "inputSchema": {"type": "object", "properties": {
                        "memory_id": {"type": "string"}
                }, "required": ["memory_id"]}},
            {"name": "list_memories", "description": "List recent memories",
                "inputSchema": {"type": "object", "properties": {
                        "category": {"type": "string", "default": ""},
                        "limit": {"type": "integer", "default": 20}
                }}},
            {"name": "memory_stats", "description": "Memory stats",
                "inputSchema": {"type": "object", "properties": {}}}
        ]
        return jsonify({"jsonrpc": "2.0", "result": {"tools": tools}, "id": req_id})

    if method == 'tools/call':
        tool_name = params.get('name', '')
        args = params.get('arguments', {})
        result = call_tool(tool_name, args)
        return jsonify({"jsonrpc": "2.0", "result": {"content": [{"type": "text", "text": 
json.dumps(result, ensure_ascii=False)}]}, "id": req_id})

    return jsonify({"jsonrpc": "2.0", "error": {"code": -32601, "message": "Method not found"}, 
"id": req_id})

def call_tool(name, args):
    conn = get_db()
    try:
        if name == 'remember':
            mem_id = f"mem_{int(time.time()*1000)}"
            now = time.time()
            conn.execute("INSERT INTO memories VALUES (?,?,?,?,?,?,?)",
                (mem_id, args['content'], args.get('category','general'),
```

```
            args.get('tags',''), args.get('source','ai'), now, now))
        conn.commit()
        return {"status": "ok", "memory_id": mem_id}

    elif name == 'recall':
        query = args['query'].lower()
        limit = args.get('limit', 5)
        category = args.get('category', '')
        sql = "SELECT * FROM memories"
        if category:
            sql += f" WHERE category = '{category}'"
        rows = conn.execute(sql + " ORDER BY created_at DESC").fetchall()
        results = []
        for row in rows:
            score = 0
            content_lower = row['content'].lower()
            tags_lower = (row['tags'] or '').lower()
            for word in query.split():
                if word in content_lower: score += 2
                if word in tags_lower: score += 1
            if score > 0:
                results.append({"memory_id": row['id'], "content": row['content'],
                    "category": row['category'], "tags": row['tags'], "score": score})
        results.sort(key=lambda x: x['score'], reverse=True)
        return results[:limit]

    elif name == 'forget':
        conn.execute("DELETE FROM memories WHERE id = ?", (args['memory_id'],))
        conn.commit()
        return {"status": "ok"}

    elif name == 'list_memories':
        limit = args.get('limit', 20)
        category = args.get('category', '')
        sql = "SELECT * FROM memories"
        if category:
            sql += f" WHERE category = '{category}'"
        rows = conn.execute(sql + f" ORDER BY created_at DESC LIMIT {limit}").fetchall()
        return [{"memory_id": r['id'], "content": r['content'],
                "category": r['category'], "tags": r['tags']} for r in rows]

    elif name == 'memory_stats':
        total = conn.execute("SELECT COUNT(*) FROM memories").fetchone()[0]
        cats = conn.execute("SELECT category, COUNT(*) as cnt FROM memories GROUP
```

```
BY category").fetchall()
                return {"total": total, "categories": {r['category']: r['cnt'] for r in cats}}

    finally:
        conn.close()
    return {"error": "Unknown tool"}

@app.route('/health', methods=['GET'])
def health():
    conn = get_db()
    count = conn.execute("SELECT COUNT(*) FROM memories").fetchone()[0]
    conn.close()
    return jsonify({"status": "ok", "name": "cloud_memory", "memories": count})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=9800)
```

注册为系统服务

```bash
sudo tee /etc/systemd/system/memory-mcp.service << 'EOF'
[Unit]
Description=Cloud Memory MCP Server
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/memory_mcp
ExecStart=/usr/bin/python3 /home/ubuntu/memory_mcp/flask_server.py
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable memory-mcp
sudo systemctl start memory-mcp
```

验证：`curl http://localhost:9800/health`

第四步：配置 HTTPS（域名+SSL）

（国内 AI 可在腾讯云一套配齐，同样的免费申请，如果是国外 AI 请看下条）

腾讯云步骤：

域名解析

在你的域名管理面板添加 A 记录：`mem.你的域名.com` → 服务器 IP

安装 Certbot + 配置 nginx

```bash
sudo apt-get install -y certbot python3-certbot-nginx
```

创建 nginx 配置 `/etc/nginx/sites-available/memory-mcp`：

```nginx
server {
    listen 80;
    server_name mem.你的域名.com;

    location / {
        proxy_pass http://127.0.0.1:9800;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

启用并申请证书：

```bash
sudo ln -s /etc/nginx/sites-available/memory-mcp /etc/nginx/sites-enabled/
```

```
sudo nginx -t && sudo systemctl reload nginx
sudo certbot --nginx -d mem.你的域名.com
```

验证：浏览器打开 `https://mem.你的域名.com/health`

第五步：连接 Claude APP 端

1. 打开 claude.ai → Settings → Connectors
2. 点 **+** → Add custom connector
3. Name：`记忆库`（随意）
4. URL：`https://mem.你的域名.com/mcp`
5. 点 Add

成功后你会看到 5 个工具：remember、recall、forget、list_memories、memory_stats。

第六步（推荐）：Cloudflare 加速

如果你的服务器在国内，国际连接可能不稳定。用 Cloudflare 做代理：

1. 注册 cloudflare.com（免费）
2. 添加你的域名
3. 把 DNS 服务器改成 Cloudflare 提供的
4. 确保 `mem` 记录的云朵图标是**橙色**（代理开启）

这样 Claude APP（Anthropic 服务器在美国）通过 Cloudflare 全球节点连接你的服务器，稳定性大幅提升。

第七步：连接 Claude Code

在项目目录的 `.mcp.json` 中添加：

```json
{
  "mcpServers": {
    "cloud-memory": {
      "type": "http",
      "url": "http://你的服务器 IP/mcp"
    }
  }
}
```

Code 端走 HTTP 直连 IP 即可，不需要 HTTPS。

常见问题

Q: APP 端报"Couldn't reach the MCP server"
A: 检查 HTTPS 证书是否有效、nginx 是否在运行、Cloudflare 代理是否开启。

Q: 记忆数据存在哪？
A: 服务器上的 SQLite 文件：`/home/ubuntu/memory_mcp/memories.db`

Q: 怎么备份？
A: 定期下载 `memories.db` 文件即可。

Q: 服务器重启后记忆还在吗？
A: 在。systemd 配置了开机自启，数据在硬盘上不会丢。

可选：部署记忆库网页管理界面

> 一个网页面板，让你在浏览器里直接查看、搜索、编辑、删除记忆。不用命令行。

效果

- 深色主题，支持语义搜索
- 按分类过滤（general/event/quote/fan_info 等）
- 直接在网页上新增、编辑、删除记忆
- 显示记忆来源（AI 存的 / 人存的）

部署步骤

1. 上传文件

把 `memory_web.py` 上传到服务器的 `/home/ubuntu/memory_mcp/` 目录下。

2. 安装依赖（如果还没装的话）

```bash
pip3 install flask chromadb
```

> 注意：网页管理界面使用 ChromaDB（支持语义搜索），和 MCP 服务端的 SQLite 是**两

套存储**。如果你想统一，需要自行修改 `memory_web.py` 把 ChromaDB 换成 SQLite。

3. 注册为系统服务

```bash
sudo tee /etc/systemd/system/memory-web.service << 'EOF'
[Unit]
Description=Memory Web UI
After=network.target

[Service]
Type=simple
User=ubuntu
WorkingDirectory=/home/ubuntu/memory_mcp
ExecStart=/usr/bin/python3 /home/ubuntu/memory_mcp/memory_web.py
Restart=always
RestartSec=5
Environment=PYTHONUNBUFFERED=1

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable memory-web
sudo systemctl start memory-web
```

4. 配置 nginx 反向代理（可选，走 HTTPS）

在 `/etc/nginx/sites-available/memory-mcp` 里加一段：

```nginx
# 记忆管理界面（走子路径或单独子域名）
server {
    listen 80;
    server_name admin-mem.你的域名.com;

    location / {
        proxy_pass http://127.0.0.1:9527;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

```

然后申请证书：
```bash
sudo nginx -t && sudo systemctl reload nginx
sudo certbot --nginx -d admin-mem.你的域名.com
```

5. 开放端口

如果不走 nginx 反代，直接访问 9527 端口的话：
- 腾讯云防火墙：添加 TCP 9527 允许
- 1Panel 防火墙（ufw）：添加 TCP 9527 允许

6. 访问

- 走 nginx 反代：`https://admin-mem.你的域名.com`
- 直接访问：`http://你的服务器 IP:9527`

安全提示

这个管理界面**没有登录验证**，任何知道地址的人都能操作你的记忆。建议：
- 不要把地址公开分享
- 或者在 nginx 层加一个基础认证（Basic Auth）：

```bash
# 生成密码文件
sudo apt-get install -y apache2-utils
sudo htpasswd -c /etc/nginx/.htpasswd 你的用户名

# 在 nginx 的 server 块里加：
# auth_basic "Memory Admin";
# auth_basic_user_file /etc/nginx/.htpasswd;
```

修改端口

默认端口 9527，如果需要改，编辑 `memory_web.py` 最后一行：
```python
app.run(host="0.0.0.0", port=9527, debug=False)
```
改成你想要的端口，然后 `sudo systemctl restart memory-web`。
```

方案 B：腾讯云 SCF 云函数部署（无服务器方案）

【本人未实践此方案，理论上可行，适合一个月一下拿不出 99r 或者 59r 的】
此方案大概一个月 30r 左右
> 适合只需要记忆库、不需要酒馆等其他服务的场景。零运维，按量计费。

优缺点对比

	云函数 SCF	轻量服务器
成本	免费额度内 0 元，之后~1-2 元/月	99 元/年（~8 元/月）or 59/年
维护	零运维，自动扩缩	需要自己管理 or AI 自主管理
能跑其他服务	❌ 只能跑函数	☑ 酒馆/Bot 等都行
启动时间	有（1-2 秒延迟）	无（秒响应）
开发复杂度	较高（无状态+COS 存储）	低（跟本地一样）

架构

```
Claude APP/Code → HTTPS → API 网关 → 云函数(SCF) → COS 对象存储(SQLite 文件)
```

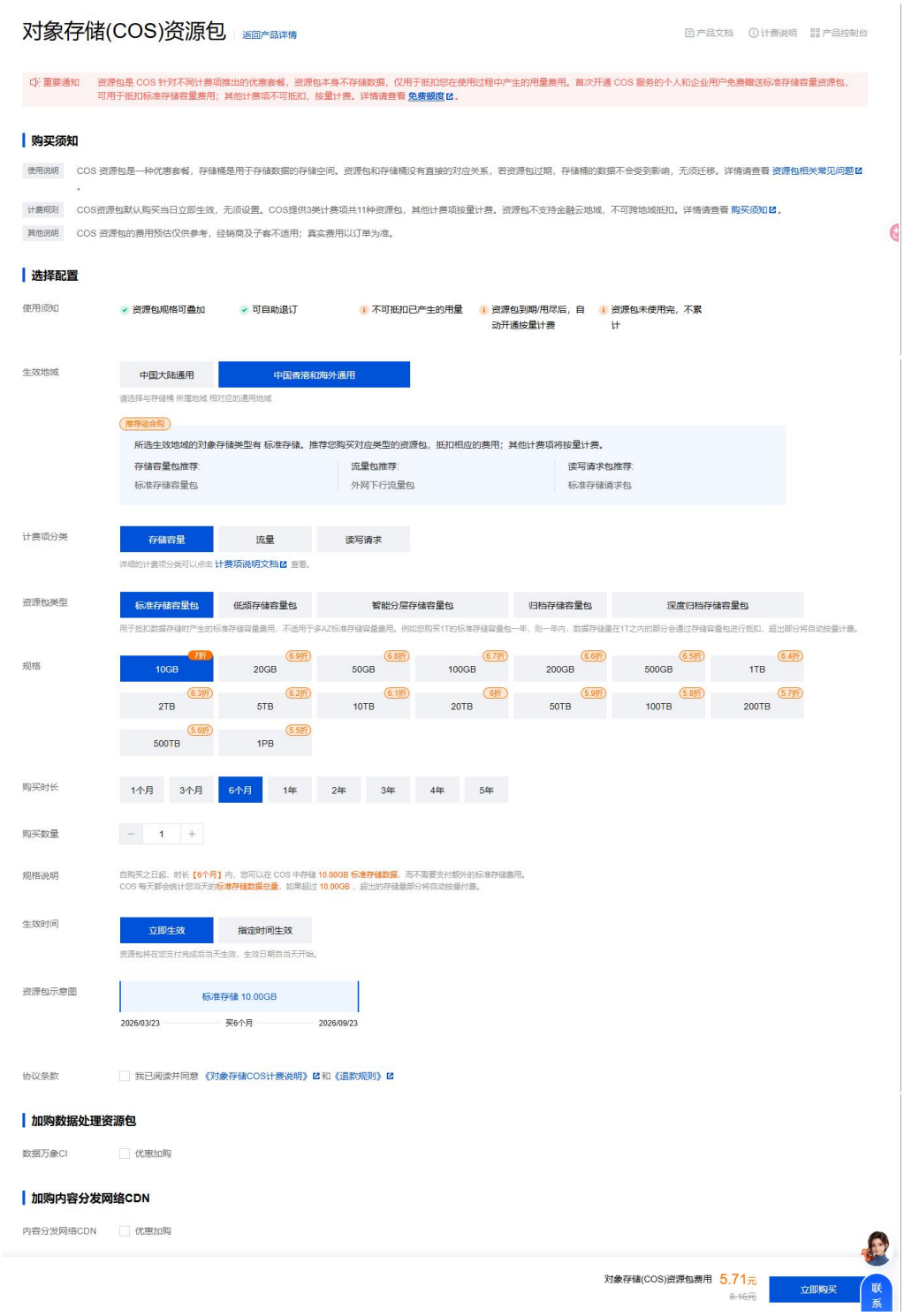
核心区别：云函数是无状态的，每次调用可能在不同的容器里跑。所以数据库不能存在本地，要存在 COS（对象存储）里，每次调用时下载→操作→上传。

第一步：开通 SCF

1. 腾讯云 → Serverless → 云函数
2. 新客试用套餐（免费 3 个月）
3. 地域选**广州**（离自己近的地方，此处用广州示例）

第二步：开通 COS

网址：https://buy.cloud.tencent.com/cos?packageType=std
1. 腾讯云 → 对象存储 COS
2. 创建一个**私有读写**的存储桶（国内 AI 选择国内即可，国外 AI 走海外通用）
3. 记下存储桶名和地域（如 `memory-bucket-123456` / `ap-guangzhou`）



第三步：编写云函数

创建 Web 函数，运行环境选 **Python 3.9+**。

入口文件 `index.py`：

```python
```

```python
"""
Cloud Memory MCP - SCF Version
每次调用从 COS 下载数据库，操作完上传回去。
"""
import json, time, sqlite3, os, tempfile
from qcloud_cos import CosConfig, CosS3Client

# === COS 配置（从环境变量读取）===
SECRET_ID = os.environ.get("COS_SECRET_ID", "")
SECRET_KEY = os.environ.get("COS_SECRET_KEY", "")
REGION = os.environ.get("COS_REGION", "ap-guangzhou")
BUCKET = os.environ.get("COS_BUCKET", "your-bucket-name")
DB_KEY = "memory_mcp/memories.db"    # COS 里的文件路径

config = CosConfig(Region=REGION, SecretId=SECRET_ID, SecretKey=SECRET_KEY)
cos_client = CosS3Client(config)

# ===  数据库操作  ===

def download_db():
    """从 COS 下载 SQLite 数据库到临时目录"""
    tmp_path = os.path.join(tempfile.gettempdir(), "memories.db")
    try:
        cos_client.download_file(BUCKET, DB_KEY, tmp_path)
    except Exception:
        #  数据库不存在，创建新的
        conn = sqlite3.connect(tmp_path)
        conn.execute("""CREATE TABLE IF NOT EXISTS memories (
            id TEXT PRIMARY KEY,
            content TEXT NOT NULL,
            category TEXT DEFAULT 'general',
            tags TEXT DEFAULT '',
            source TEXT DEFAULT 'ai',
            created_at REAL,
            updated_at REAL
        )""")
        conn.commit()
        conn.close()
    return tmp_path

def upload_db(tmp_path):
    """操作完上传回 COS"""
    cos_client.upload_file(BUCKET, DB_KEY, tmp_path)
```

```
def get_db(path):
    conn = sqlite3.connect(path)
    conn.row_factory = sqlite3.Row
    return conn

# === MCP 工具定义 ===

TOOLS = [
    {"name": "remember", "description": "Save a memory",
     "inputSchema": {"type": "object", "properties": {
        "content": {"type": "string"},
        "category": {"type": "string", "default": "general"},
        "tags": {"type": "string", "default": ""},
        "source": {"type": "string", "default": "ai"}
     }, "required": ["content"]}},
    {"name": "recall", "description": "Search memories by keyword",
     "inputSchema": {"type": "object", "properties": {
        "query": {"type": "string"},
        "limit": {"type": "integer", "default": 5},
        "category": {"type": "string", "default": ""}
     }, "required": ["query"]}},
    {"name": "forget", "description": "Delete a memory",
     "inputSchema": {"type": "object", "properties": {
        "memory_id": {"type": "string"}
     }, "required": ["memory_id"]}},
    {"name": "list_memories", "description": "List recent memories",
     "inputSchema": {"type": "object", "properties": {
        "category": {"type": "string", "default": ""},
        "limit": {"type": "integer", "default": 20}
     }}},
    {"name": "memory_stats", "description": "Memory statistics",
     "inputSchema": {"type": "object", "properties": {}}}
]

# ===  工具执行  ===

def call_tool(name, args, db_path):
    conn = get_db(db_path)
    need_upload = False
    try:
        if name == "remember":
            mem_id = f"mem_{int(time.time() * 1000)}"
            now = time.time()
            conn.execute("INSERT INTO memories VALUES (?,?,?,?,?,?,?)",
```

```
            (mem_id, args["content"], args.get("category", "general"),
             args.get("tags", ""), args.get("source", "ai"), now, now))
        conn.commit()
        need_upload = True
        return {"status": "ok", "memory_id": mem_id}, need_upload

    elif name == "recall":
        query = args["query"].lower()
        limit = args.get("limit", 5)
        category = args.get("category", "")
        sql = "SELECT * FROM memories"
        params = []
        if category:
            sql += " WHERE category = ?"
            params.append(category)
        rows = conn.execute(sql + " ORDER BY created_at DESC", params).fetchall()
        results = []
        for row in rows:
            score = 0
            content_lower = row["content"].lower()
            tags_lower = (row["tags"] or "").lower()
            for word in query.split():
                if word in content_lower: score += 2
                if word in tags_lower: score += 1
            if score > 0:
                results.append({"memory_id": row["id"], "content": row["content"],
                    "category": row["category"], "tags": row["tags"], "score": score})
        results.sort(key=lambda x: x["score"], reverse=True)
        return results[:limit], need_upload

    elif name == "forget":
        conn.execute("DELETE FROM memories WHERE id = ?", (args["memory_id"],))
        conn.commit()
        need_upload = True
        return {"status": "ok"}, need_upload

    elif name == "list_memories":
        limit = args.get("limit", 20)
        category = args.get("category", "")
        sql = "SELECT * FROM memories"
        params = []
        if category:
            sql += " WHERE category = ?"
            params.append(category)
```

```
            rows = conn.execute(sql + f" ORDER BY created_at DESC LIMIT ?",
                                params + [limit]).fetchall()
            return [{"memory_id": r["id"], "content": r["content"],
                    "category": r["category"], "tags": r["tags"]} for r in rows], need_upload

        elif name == "memory_stats":
            total = conn.execute("SELECT COUNT(*) FROM memories").fetchone()[0]
            cats = conn.execute("SELECT category, COUNT(*) as cnt FROM memories GROUP
BY category").fetchall()
            return {"total": total, "categories": {r["category"]: r["cnt"] for r in cats}},
need_upload

    finally:
        conn.close()
    return {"error": "Unknown tool"}, False

# === SCF 入口 ===

def main_handler(event, context):
    """腾讯云 SCF Web 函数入口"""
    # 解析请求
    method = event.get("httpMethod", "GET")
    path = event.get("path", "/")
    body = event.get("body", "")

    # Health check
    if method == "GET" and path == "/health":
        db_path = download_db()
        conn = get_db(db_path)
        count = conn.execute("SELECT COUNT(*) FROM memories").fetchone()[0]
        conn.close()
        return {
            "statusCode": 200,
            "headers": {"Content-Type": "application/json"},
            "body": json.dumps({"status": "ok", "name": "cloud-memory-scf", "memories":
count})
        }

    # MCP endpoint
    if method == "POST" and path == "/mcp":
        try:
            data = json.loads(body) if isinstance(body, str) else body
        except:
            return {"statusCode": 400, "body": json.dumps({"error": "Invalid JSON"})}
```

```
        req_method = data.get("method", "")
        params = data.get("params", {})
        req_id = data.get("id")

        # Initialize
        if req_method == "initialize":
            result = {
                "protocolVersion": "2024-11-05",
                "capabilities": {"tools": {}},
                "serverInfo": {"name": "cloud-memory-scf", "version": "1.0.0"}
            }
            return {"statusCode": 200, "headers": {"Content-Type": "application/json"},
                    "body": json.dumps({"jsonrpc": "2.0", "result": result, "id": req_id})}

        if req_method == "notifications/initialized":
            return {"statusCode": 204}

        # Tools list
        if req_method == "tools/list":
            return {"statusCode": 200, "headers": {"Content-Type": "application/json"},
                    "body": json.dumps({"jsonrpc": "2.0", "result": {"tools": TOOLS}, "id": req_id})}

        # Tool call
        if req_method == "tools/call":
            db_path = download_db()
            tool_name = params.get("name", "")
            args = params.get("arguments", {})
            result, need_upload = call_tool(tool_name, args, db_path)
            if need_upload:
                upload_db(db_path)
            return {"statusCode": 200, "headers": {"Content-Type": "application/json"},
                    "body": json.dumps({"jsonrpc": "2.0", "result": {
                        "content": [{"type": "text", "text": json.dumps(result, ensure_ascii=False)}]
                    }, "id": req_id})}

    return {"statusCode": 404, "body": "Not Found"}
```

第四步：配置环境变量

在 SCF 控制台设置函数的环境变量：

变量名	值
COS_SECRET_ID	你的腾讯云 SecretId
COS_SECRET_KEY	你的腾讯云 SecretKey
COS_REGION	ap-guangzhou
COS_BUCKET	你的存储桶名

第五步：配置 API 网关触发器

1. 在云函数页面 → 触发管理 → 创建触发器
2. 选择 **API 网关触发器**
3. 新建 API 服务
4. 创建完成后会得到一个 URL，如：`https://xxx.gz.apigw.tencentcs.com/release/`

第六步：连接 Claude

Claude APP
Settings → Connectors → Add custom connector：
- URL: `https://xxx.gz.apigw.tencentcs.com/release/mcp`

Claude Code
`.mcp.json`:
```json
{
  "mcpServers": {
    "cloud-memory": {
      "type": "http",
      "url": "https://xxx.gz.apigw.tencentcs.com/release/mcp"
    }
  }
}
```

依赖打包

SCF 需要把依赖一起打包上传。创建部署包：

```bash
mkdir deploy && cd deploy
pip install cos-python-sdk-v5 -t .
cp ../index.py .
zip -r ../memory-mcp-scf.zip .
```

上传 `memory-mcp-scf.zip` 到 SCF 即可。

注意事项

- **冷启动**：首次调用或长时间不用后会有 1-2 秒延迟（下载数据库）
- **并发写入**：多个请求同时写入可能冲突。个人使用基本不会遇到
- **免费额度**：个人标准版每月 50 万次调用，记忆库日常使用绰绰有余
- **API 网关**：可能需要额外配置 HTTPS 和自定义域名
- **成本**：免费试用 3 个月，之后 COS 约 12 元/月 + SCF 按量计费（极低）